Name: P. Sai Charan Tej
Reg_No:192465048

Experiment 1: Live Acquisition of Volatile Memory

Aim:

To perform live acquisition of volatile memory (RAM) from a running Windows system while maintaining evidence integrity.

Tools Used:

- WinPmem
- Command Prompt (Administrator)

Procedure:

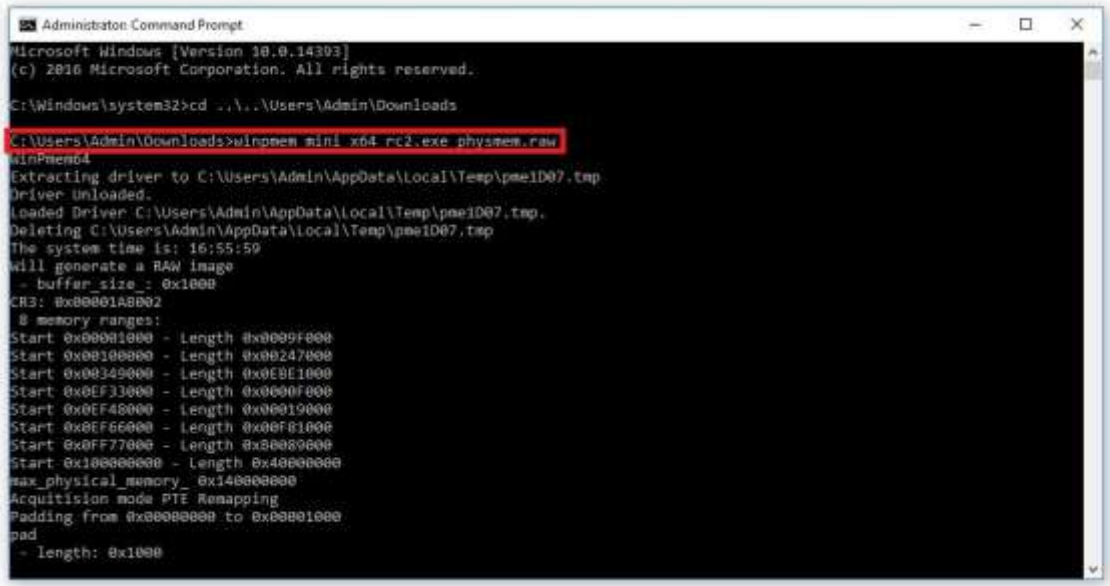
- Download and extract the WinPmem tool.
- Open Command Prompt as Administrator.
- Navigate to the WinPmem folder.
- Run the command: winpmem.exe memory_dump.raw
- Wait until the memory acquisition process completes.
- Verify that the file 'memory_dump.raw' has been created successfully.

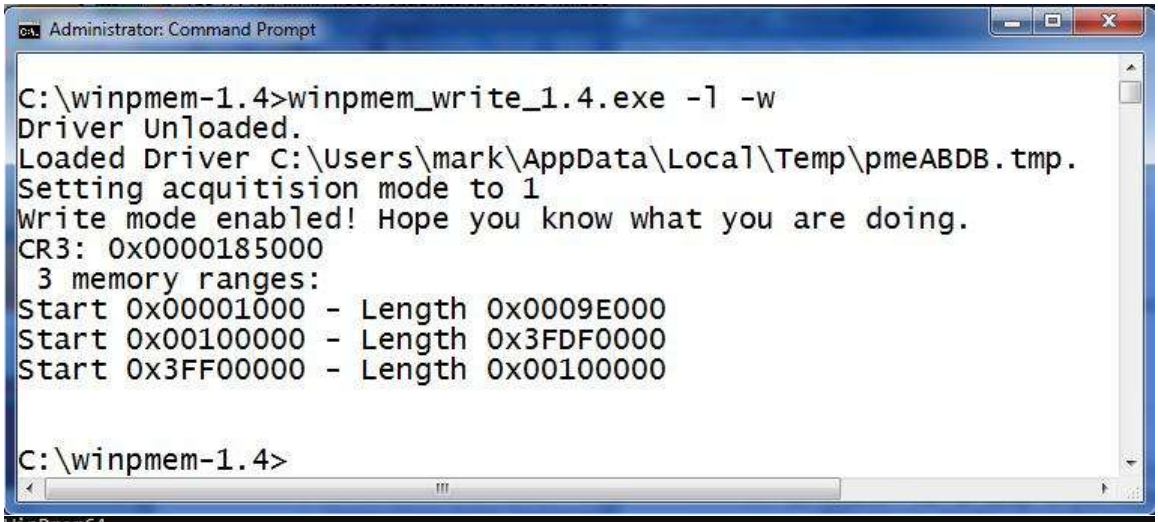
Expected Output:

A forensic RAM image (memory_dump.raw) is successfully created for further investigation.

Result:

The volatile memory of the running system was successfully acquired while maintaining forensic integrity.






```
WinPmem64
Extracting driver to C:\Users\kkari\AppData\Local\Temp\pme3BAF.tmp
Driver Unloaded.
Loaded Driver C:\Users\kkari\AppData\Local\Temp\pme3BAF.tmp.
Deleting C:\Users\kkari\AppData\Local\Temp\pme3BAF.tmp
The system time is: 00:55:42
Will generate a RAW image
 - buffer_size_: 0x1000
CR3: 0x00001AA002
 4 memory ranges:
Start 0x00001000 - Length 0x0009E000
Start 0x00100000 - Length 0x00002000
Start 0x00103000 - Length 0xDFEED000
Start 0x100000000 - Length 0x59400000
max_physical_memory_ 0x159400000
Acquisition mode PTE Remapping
Padding from 0x00000000 to 0x00001000
pad
 - length: 0x1000

00% 0x00000000 .
copy_memory
 - start: 0x1000
 - end: 0x9f000
```

Experiment 2: Create a Bit-Stream (Forensic) Image and Verify Integrity

Aim

To create a bit-stream forensic image of a sample file and verify its integrity using MD5 and SHA-256 hash values in Kali Linux.

Tools Required

- Kali Linux
- Terminal
- dd
- md5sum
- sha256sum

Procedure

1. Create a working directory using: mkdir CSA61

2. Move into the directory: cd CSA61

3. Verify the current path using: pwd

4. Create a sample evidence file: echo "Digital Forensics Practical - CSA61" > evidence.txt

5. Verify the file using: ls -l

6. Create a bit-stream image: dd if=evidence.txt of=evidence.dd bs=4K status=progress

7. Verify the created image using: ls -lh

8. Generate the MD5 hash of evidence.txt using: md5sum evidence.txt

9. Generate the MD5 hash of evidence.dd using: md5sum evidence.dd

10. Generate the SHA-256 hash of evidence.txt using: sha256sum evidence.txt

11. Generate the SHA-256 hash of evidence.dd using: sha256sum evidence.dd

Observations

File	MD5	SHA-256
evidence.txt	01b8eee9bee5f64214ee126ed81529f1	dd5fae6c5d976c8629844d692e1ce6d0d70a9585a194e2e77a8c1c48411bd73f
evidence.dd	01b8eee9bee5f64214ee126ed81529f1	dd5fae6c5d976c8629844d692e1ce6d0d70a9585a194e2e77a8c1c48411bd73f

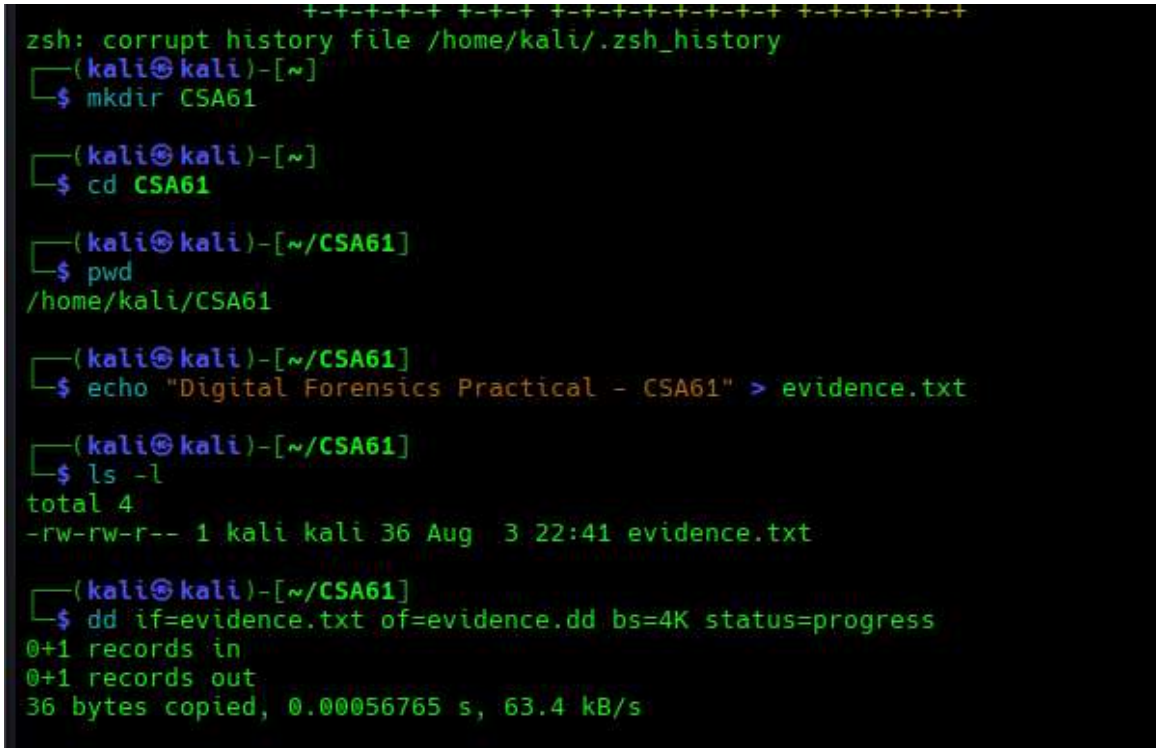
Result

The bit-stream image (evidence.dd) was successfully created using the dd command. The MD5 and SHA-256 hash values of the original evidence file and the forensic image matched, confirming that the integrity of the evidence was preserved.

Screenshots

Step 1: Create directory and navigate

Step 2: Create evidence file



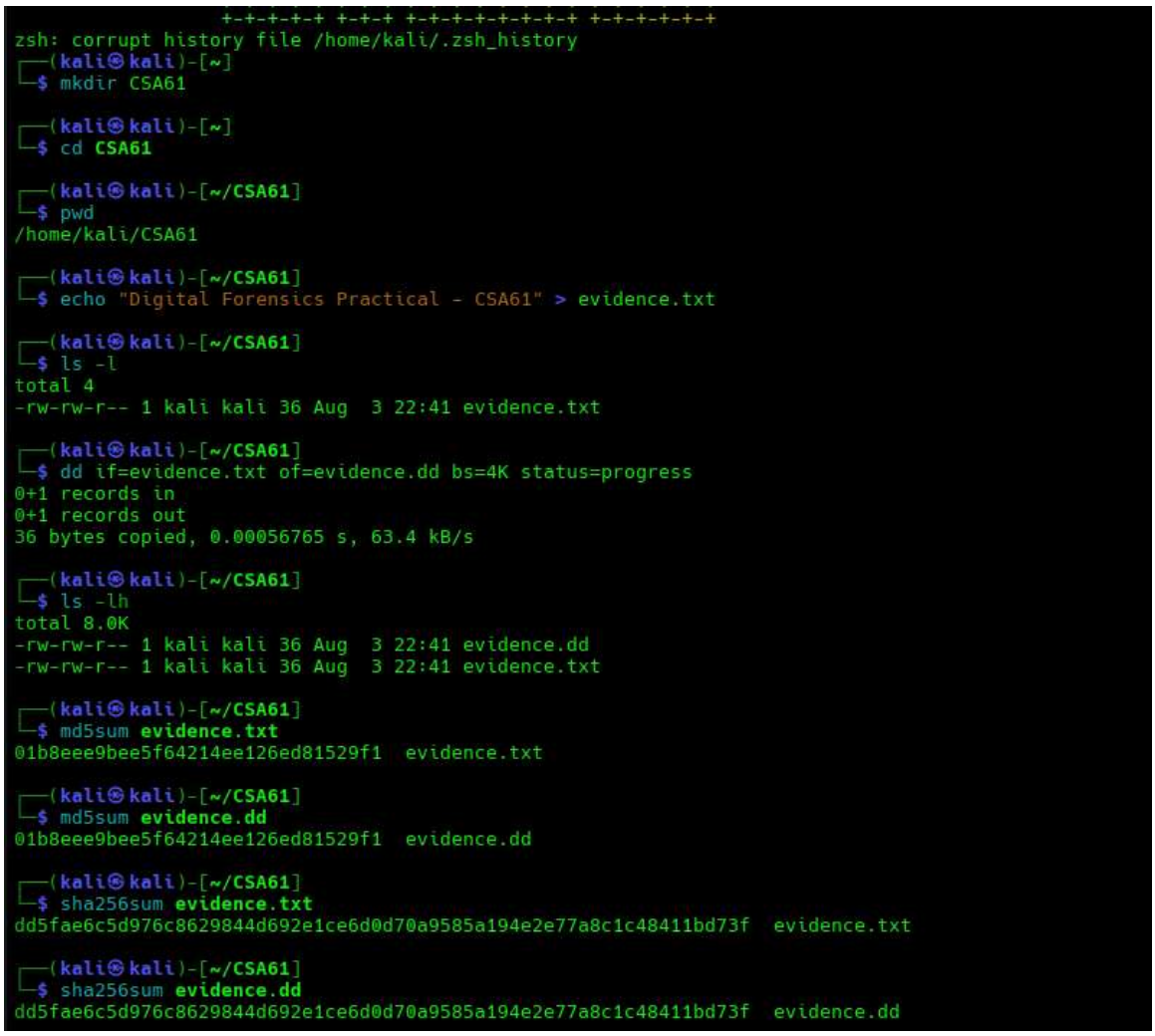
Step 3: Create forensic image using dd

```
zsh: corrupt history file /home/kali/.zsh_history
┌──(kali㉿kali)-[~]
└─$ mkdir CSA61

┌──(kali㉿kali)-[~]
└─$ cd CSA61

┌──(kali㉿kali)-[~/CSA61]
└─$ pwd
/home/kali/CSA61

┌──(kali㉿kali)-[~/CSA61]
└─$ echo "Digital Forensics Practical - CSA61" > evidence.txt

┌──(kali㉿kali)-[~/CSA61]
└─$ ls -l
total 4
-rw-rw-r-- 1 kali kali 36 Aug  3 22:41 evidence.txt

┌──(kali㉿kali)-[~/CSA61]
└─$ dd if=evidence.txt of=evidence.dd bs=4K status=progress
0+1 records in
0+1 records out
36 bytes copied, 0.00056765 s, 63.4 kB/s

┌──(kali㉿kali)-[~/CSA61]
└─$ ls -lh
total 8.0K
-rw-rw-r-- 1 kali kali 36 Aug  3 22:41 evidence.dd
-rw-rw-r-- 1 kali kali 36 Aug  3 22:41 evidence.txt
```

Step 4: Verify image and MD5 hashes

```
zsh: corrupt history file /home/kali/.zsh_history
┌──(kali㉿kali)-[~]
└─$ mkdir CSA61

┌──(kali㉿kali)-[~]
└─$ cd CSA61

┌──(kali㉿kali)-[~/CSA61]
└─$ pwd
/home/kali/CSA61

┌──(kali㉿kali)-[~/CSA61]
└─$ echo "Digital Forensics Practical - CSA61" > evidence.txt

┌──(kali㉿kali)-[~/CSA61]
└─$ ls -l
total 4
-rw-rw-r-- 1 kali kali 36 Aug  3 22:41 evidence.txt

┌──(kali㉿kali)-[~/CSA61]
└─$ dd if=evidence.txt of=evidence.dd bs=4K status=progress
0+1 records in
0+1 records out
36 bytes copied, 0.00056765 s, 63.4 kB/s

┌──(kali㉿kali)-[~/CSA61]
└─$ ls -lh
total 8.0K
-rw-rw-r-- 1 kali kali 36 Aug  3 22:41 evidence.dd
-rw-rw-r-- 1 kali kali 36 Aug  3 22:41 evidence.txt

┌──(kali㉿kali)-[~/CSA61]
└─$ md5sum evidence.txt
01b8eee9bee5f64214ee126ed81529f1  evidence.txt

┌──(kali㉿kali)-[~/CSA61]
└─$ md5sum evidence.dd
01b8eee9bee5f64214ee126ed81529f1  evidence.dd
```

Step 5: Verify SHA-256 hashes



Experiment 3: File Signature (Magic Number) Analysis

Aim:

To identify the original file type of files with altered extensions using magic number analysis.

Tool Used:

- HxD Hex Editor

Procedure:

- Open the suspicious file in HxD Hex Editor.
- Observe the first hexadecimal bytes (magic number).
- Compare the bytes with known file signatures.

- Identify the original file type.
- Rename the file using the correct extension.

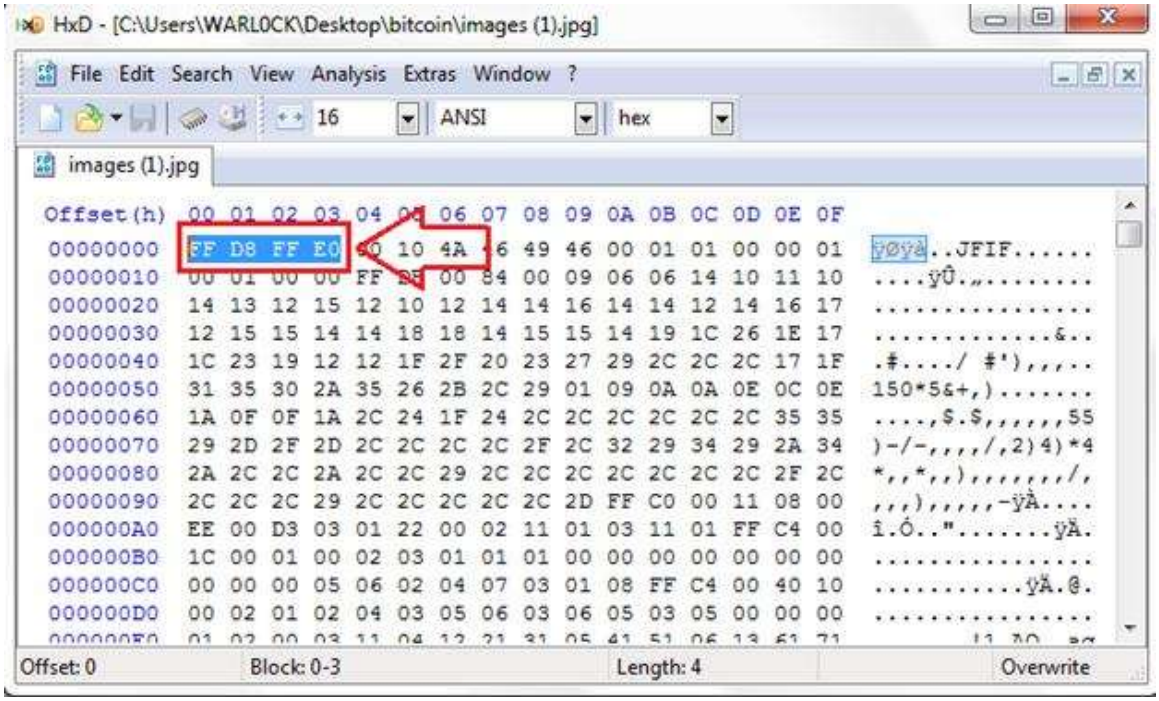
Magic Number	Original File Type
FF D8 FF	JPEG (.jpg)
89 50 4E 47	PNG (.png)
25 50 44 46	PDF (.pdf)
50 4B 03 04	ZIP / DOCX

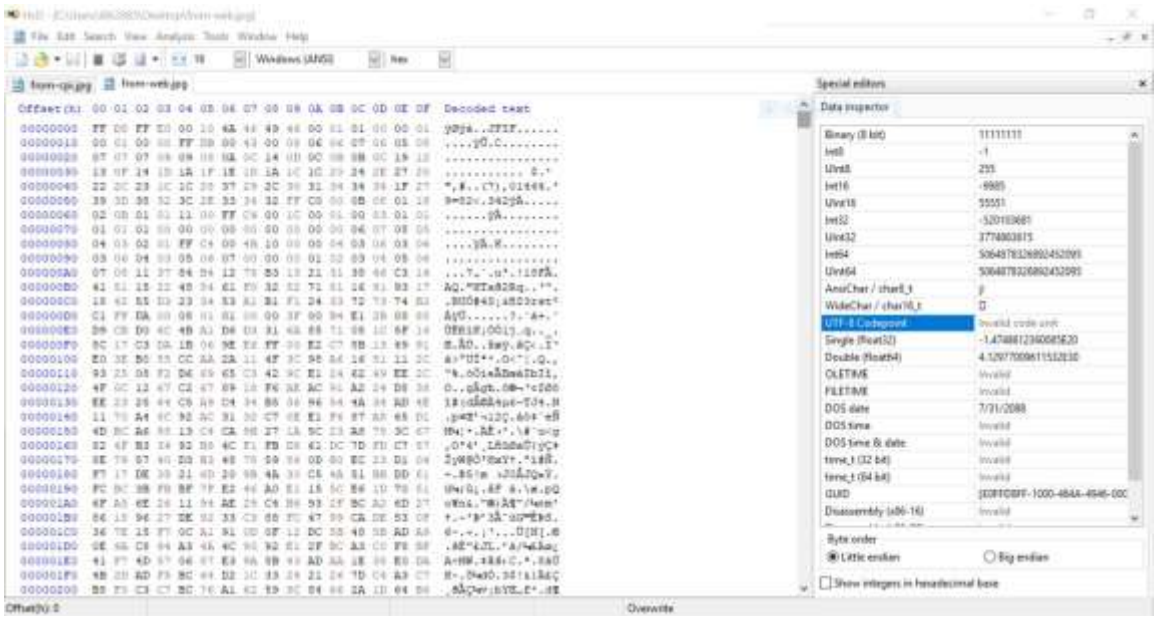
Expected Output:

The original file type is identified correctly based on the file signature.

Result:

Magic number analysis successfully identified the actual file type despite the modified extension.





HxD - [C:\Bug\PiO\2019-10\HDMI kabel.jpg]

File Edit Search View Analysis Extras Window ?

16 | ANSI | hex

HDMI kabel.jpg

```
Offset(h)  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

00000000   FF D8 FF E0 00 10 4A 46 49 46 00 01 01 00 00 01  ˙Ř˙ŕ..JFIF.....
00000010   00 01 00 00 FF DB 00 43 00 0B 08 08 08 08 08 0B  ....˙Ű.C........
00000020   08 08 0B 10 0B 09 0B 10 13 0E 0B 0B 0E 13 16 12  ................
00000030   12 13 12 12 16 15 11 13 12 12 13 11 15 15 19 1A  ................
00000040   1B 1A 19 15 21 21 24 24 21 21 30 2F 2F 2F 30 36  ....!!$$!!0///06
00000050   36 36 36 36 36 36 36 36 36 FF DB 00 43 01 0C 0B  666666666˙Ű.C...
00000060   0B 0C 0D 0C 0F 0D 0D 0F 13 0E 0E 0E 13 14 0E 0F  ................
00000070   0F 0E 14 1A 12 12 14 12 12 1A 22 18 15 15 15 15  ..........".....
00000080   18 22 1E 20 1B 1B 1B 20 1E 25 25 22 22 25 25 2F  .". ... .%%""%%/
00000090   2F 2C 2F 2F 36 36 36 36 36 36 36 36 36 36 FF C0  /,//6666666666˙Ŕ
000000A0   00 11 08 05 DC 05 DC 03 01 22 00 02 11 01 03 11  ....Ü.Ü.."......
000000B0   01 FF C4 00 1F 00 00 01 05 01 01 01 01 01 01 00  .˙Ä.............
000000C0   00 00 00 00 00 00 00 01 02 03 04 05 06 07 08 09  ................
000000D0   0A 0B FF C4 00 B5 10 00 02 01 03 03 02 04 03 05  ..˙Ä.µ..........
000000E0   05 04 04 00 00 01 7D 01 02 03 00 04 11 05 12 21  ......}........!
000000F0   31 41 06 13 51 61 07 22 71 14 32 81 91 A1 08 23  1A..Qa."q.2.‘˙.#
00000100   42 B1 C1 15 52 D1 F0 24 33 62 72 82 09 0A 16 17  B±Á.RŃđ$3br,....
00000110   18 19 1A 25 26 27 28 29 2A 34 35 36 37 38 39 3A  ...%&'()*456789:
00000120   43 44 45 46 47 48 49 4A 53 54 55 56 57 58 59 5A  CDEFGHIJSTUVWXYZ
00000130   63 64 65 66 67 68 69 6A 73 74 75 76 77 78 79 7A  cdefghijstuvwxyz
```

Offset: 6 | Block: 6-9 | Length: 4 | Overwrite